Open in app ↗

Medium Search Write A

Get unlimited access to the best of Medium for less than $1/week. Become a member

Expedia Group Technology

EXPEDIA GROUP TECHNOLOGY — DATA

Apache Spark Structured Streaming — First Streaming Example (1 of 6)

Using a scalable and fault-tolerant stream processing engine



Neeraj Bhadani · Follow

Published in Expedia Group Technology · 5 min read · Jan 28, 2021

261 3

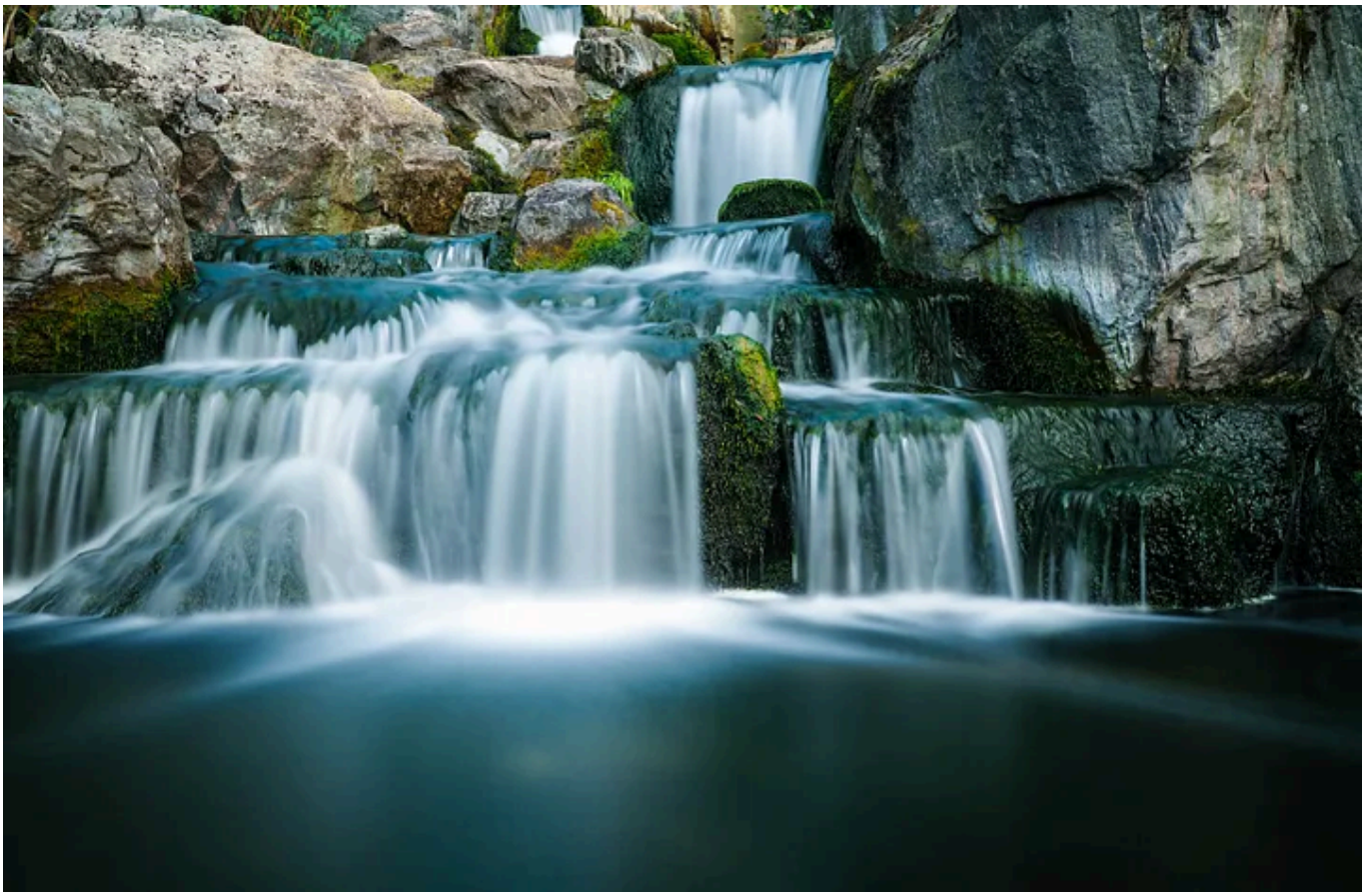


Photo by Mike Lewis HeadSmart Media on Unsplash

In this blog series, we discuss Apache Spark™ Structured Streaming. We cover components of Apache Spark Structured Streaming and play with examples to understand them.

You may also be interested in some of my other posts on Apache Spark.

- Deep Dive into Apache Spark DateTime Functions
- Working with JSON in Apache Spark
- Deep Dive into Apache Spark Window Functions
- Deep Dive into Apache Spark Array Functions
- Start Your Journey with Apache Spark

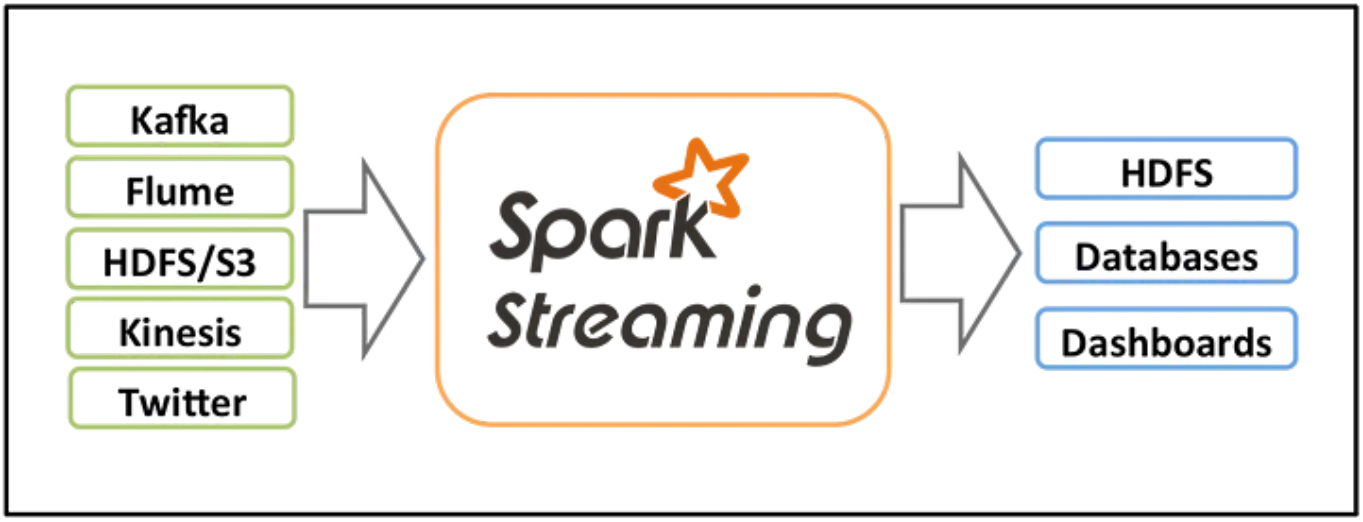


Note: We use a Scala API in this blog series. For easy reference, Scala files are available on ***GitHub****.*

Let's get started with our journey on Apache Spark Structured Streaming.

Overview

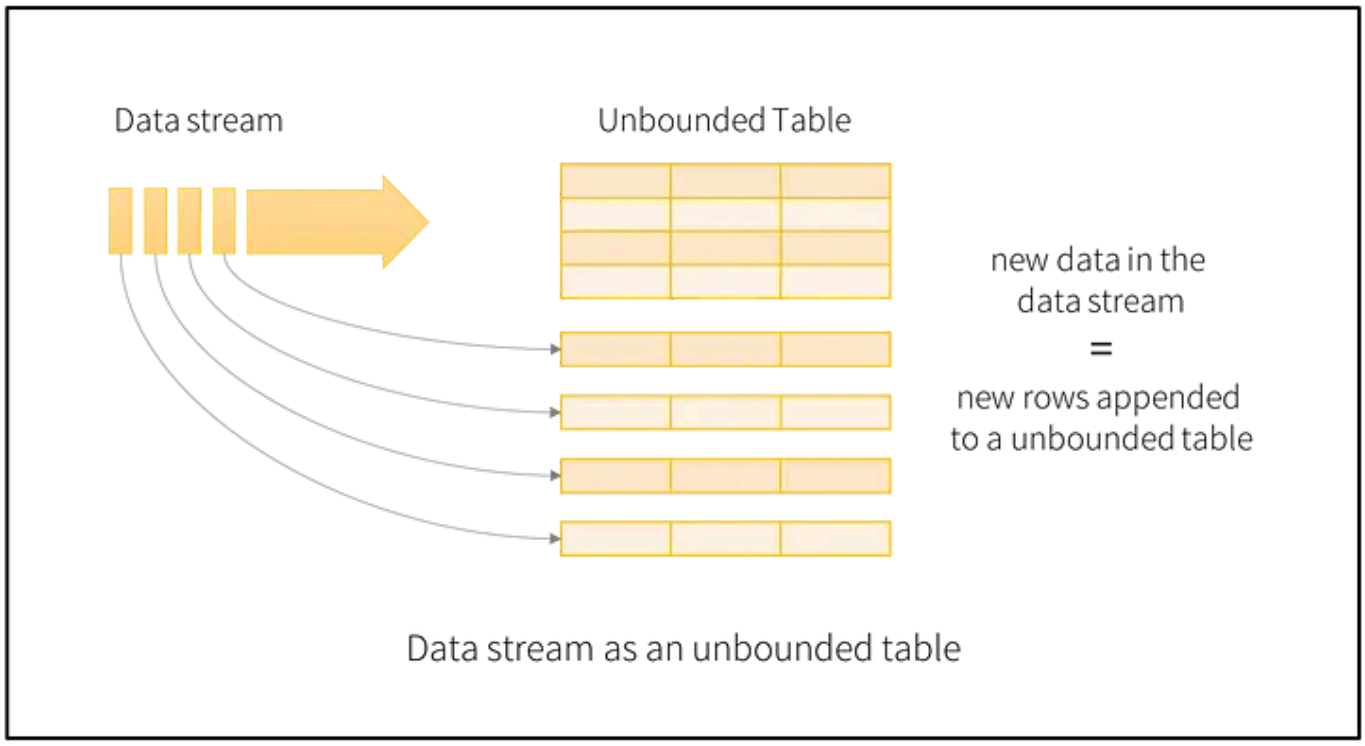
Working with streaming data is a little different from working with batch data. With streaming data, we will never have complete data for analysis, as data is continuously coming in. Apache Spark provides a streaming API to analyze streaming data in pretty much the same way we work with batch data. Apache Spark Structured Streaming is built on top of the Spark-SQL API to leverage its optimization. Spark Streaming is a processing engine to process data in real-time from sources and output data to external storage systems.



Reference

Spark Streaming has 3 major components: input sources, streaming engine, and sink. Input sources generate data like Kafka, Flume, HDFS/S3, etc. Spark Streaming engine processes incoming data from various input sources. Sinks store processed data from Spark Streaming engine like HDFS, relational databases, or NoSQL datastores.

Let's conceptualise Spark Streaming data as an unbounded table where new data will always be appended at the end of the table.



Reference

Spark will process data in micro-batches which can be defined by triggers. For example, let's say we define a trigger as `1 second`, this means Spark will create micro-batches every second and process them accordingly. We will discuss triggers in a separate blog.

Output modes

After processing the streaming data, Spark needs to store it somewhere on persistent storage. Spark uses various output modes to store the streaming data.

- **Append Mode**: In this mode, Spark will output only newly processed rows since the last trigger.
- **Update Mode**: In this mode, Spark will output only updated rows since the last trigger. If we are not using aggregation on streaming data (meaning previous records can't be updated) then it will behave similarly to append mode.
- **Complete Mode**: In this mode, Spark will output all the rows it has processed so far.

Our first streaming example using `rate` source

Now let's get our hands dirty with our first Spark Streaming example using `rate` source and `console` sink. Rate source will auto-generate data which we will then print onto a console.

Import libraries

Let's first import the required libraries.

Create Spark session

Now let's create the `sparkSession` and set the logging level to `Error` to avoid the `Warning` and `INFO` logs.

Create streaming DataFrame

Let's create our first Spark Streaming DataFrame using `rate` source. Here we have specified the format as `rate` and specified `rowsPerSecond = 1` to generate `1` row for each micro-batch and load the data into `initDF` streaming DataFrame. Also, we check if the `initDF` is a streaming DataFrame or not.

Output should read as follows:

```
Streaming DataFrame : true
```

Basic transformation

Perform a basic transformation on `initDF` to generate another column `result` by just adding `1` to column `value` :

We created a derived column `result` from an existing column `value` in a very similar way to creating one in a batch DataFrame. You can find various operations on batch DataFrame in this blog.

Output to console

Let's try to print the contents of a streaming DataFrame to console. Here, we use `append` output mode to output only newly generated data and format as `console` to print the result on the console.

This output should resemble the following:

```
-------------------------------------------
Batch: 1
-------------------------------------------
+-----------------------+-----+------+
|timestamp              |value|result|
+-----------------------+-----+------+
|2020-12-28 11:40:37.867|0    |1     |
+-----------------------+-----+------+

-------------------------------------------
Batch: 2
-------------------------------------------
+-----------------------+-----+------+
|timestamp              |value|result|
+-----------------------+-----+------+
|2020-12-28 11:40:38.867|1    |2     |
+-----------------------+-----+------+
```

Here we printed the result of 2 micro-batches which contain columns `timestamp`, `value` and the generated `result` column. We have only one record per second (check timestamps above) as specified in `initDF` and `value` was generated automatically. Also, in each batch we only get newly generated data since we have used the `append` output mode.

We have successfully written our first spark streaming application!

- - - - - -

The next post in this series covers input sources of different types.

I hope you enjoyed this overview of Spark Streaming! For easy reference, you can find the complete code on GitHub.

Here are other blogs on Apache Spark Structured Streaming series.

- Apache Spark Structured Streaming — Input Sources
- Apache Spark Structured Streaming — Output Sinks
- Apache Spark Structured Streaming — Checkpoints and Triggers
- Apache Spark Structured Streaming — Operations
- Apache Spark Structured Streaming — Watermarking

Reference

- https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

This blog post is provided for educational purposes only. Certain scenarios may utilize fictitious names and test data for illustrative purposes. The content is intended as a contribution to the technological community at large and may not necessarily reflect any technology currently being implemented or planned for implementation at Expedia Group, Inc.

Programming | Software Engineering | Data Science | Streaming | Technology

Published in Expedia Group Technology

5K Followers · Last published Feb 25, 2025

Follow

Stories from the Expedia Group Technology teams



Written by Neeraj Bhadani

695 Followers · 25 Following

Follow

Data Science and cloud computing enthusiast

Responses (3)

A Akhedkar

What are your thoughts?



Neeraj Bhadani Author
Apr 29, 2021

Thanks Kefei. I have changed the code.

Reply



Chris Bao
Aug 26, 2022

Your series on Spark is great.

Reply



Kefei Mo
Apr 29, 2021

Nice blog.

I think you mean resultDF.writeStream (not initDF.writeStream)

resultDF

```
.writeStream

.outputMode("append")

.option("truncate", false)... more
```

Reply

More from Neeraj Bhadani and Expedia Group Technology

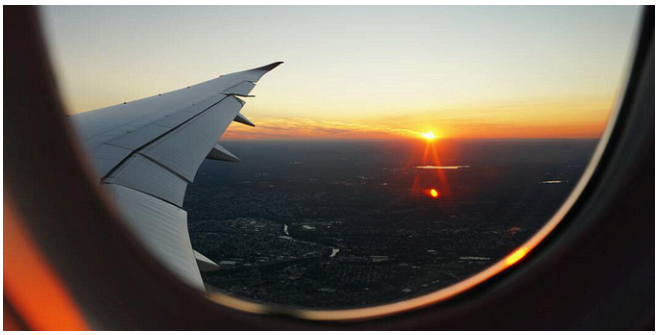


In Expedia Group Technology by Neeraj Bhadani

Deep dive into Apache Spark Window Functions

Window functions operate on groups of data and return values for each record or group

Jun 25, 2020 557 4



In Expedia Group Technology by Ritesh Kapoor

Onion Architecture

Let's slice it like a Pro

Jul 12, 2022 319 8



In Expedia Group Technolo... by Christine Mathies...

A Short Introduction to Apache Iceberg

The data lake team at Expedia Group starts working with table formats, adds Hive...

Jan 26, 2021 662 3



In Expedia Group Technology by Neeraj Bhadani

Deep Dive into Apache Spark Array Functions

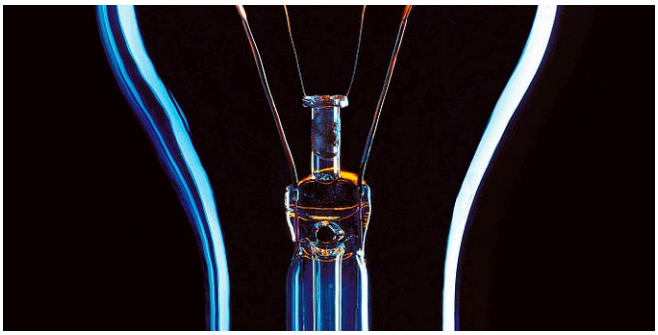
A practical guide to using array functions

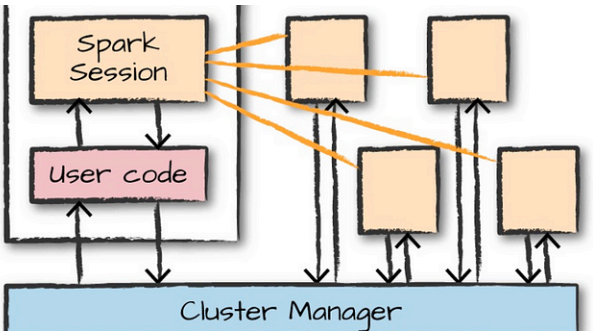
Jul 16, 2020 201 3

See all from Neeraj Bhadani

See all from Expedia Group Technology

Recommended from Medium





In Towards Data Engineering by Ritam Mukherjee

Building Real-Time Recommendations with Spark,...

Ever wondered how online stores know exactly what you'd like to buy next

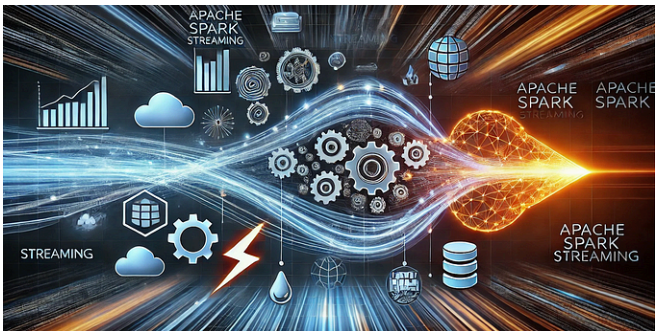
Nov 30, 2024 80

In Towards Dev by Prem Vishnoi(cloudvala)

Apache Spark Architecture :A Deep Dive into Big Data Processing

Agenda

Feb 6 90 1

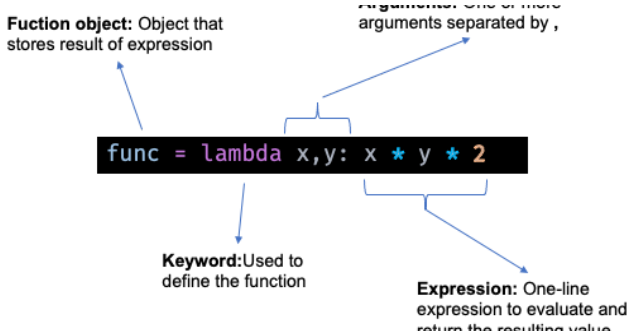


Ashutosh Kumar

Apache Spark Streaming—All you need to know...

If you are working in the Analytics, Machine Learning or Data Engineering space, you...

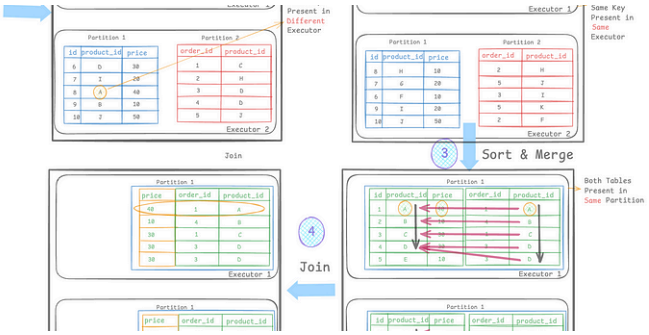
Jan 2 6



Martin Jurado Pedroza

Lambda functions and built-in functions (Python and Pyspark)

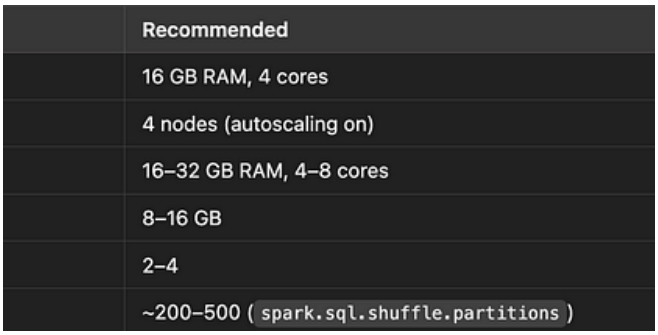
Nov 23, 2024 1



Shantanu Tripathi

Sort Merge Join: Visual Flow inside Executors

Learn the default join strategy used by Spark through visuals.



Kaizen

How to Read a 100GB File in PySpark Without Breaking Your...

Working with 100GB+ datasets is a reality in modern data engineering. But reading a...



See more recommendations